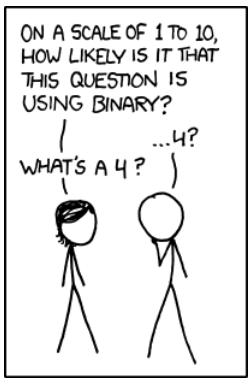
CS51 - Assignment 3
Bitwise Puzzles

Due: September, 18th at 11:59pm



https://xkcd.com/953/

For this assignment, we will:

- Become more comfortable with the binary representation of integers.
- Practice binary arithmetic by implementing various bitwise operations from scratch in Python.
- Practice writing Python code that is organized in functions.
- Practice testing your Python code.

1 Background

The goal of this assignment is to increase your familiarity with the binary representation of information, specifically integer numbers. You are asked to complete a series of eight logical puzzles that apply bitwise operations, that is, operations on the individual bits of the binary representation of integers. These puzzles will be solved in Python, but you will benefit from working on pen and paper first. You should also review the slides on Binary Arithmetic & Representing Information and De Morgan's Laws from the slides on Boolean Algebra.

This is a **group assignment**. You must work with a partner from the same lab. Both partners should work together at all times rather than dividing the puzzles.

2 The Eight Puzzles

We have provided you with a file, `assignment3.py`, that provides skeleton code for the eight puzzles you will solve with your partner. Start by adding your names on the allocated space.

2.1 Functions in Python

The eight puzzles are organized into eight *functions*. If you are not familiar or need a refresher, you can think of functions as reusable pieces of code that take input, do something with that input, and give back an output. Functions allow us to break big problems into smaller modular pieces where code is written once but can be reused many times, avoiding repetition and errors.

Here's a crash course on functions, in Python but please do not hesitate to ask staff questions if you are still unclear about them.

The syntax of defining a function in Python follows this recipe:

```
def greet(name):
    return "Hello, " + name
```

- `def` – tells Python that you're defining a function. You always start with this keyword when defining a function.
- `greet` – is the name of the function. You choose an appropriate name that captures what the function does.
- `name` – is the input passed to the function. We call the input the *parameter*. A function can take more than one parameter (or sometimes even none). The parameter(s) are enclosed in parentheses and the closing parenthesis is always followed by `:`.
- `return` – gives back the result. The function we just defined receives a name, and the only work that it does is that it returns the string "Hello, " and whatever name was passed as input. But it could have been the case that *before* the `return` keyword, we had more lines of code because we had to do more complicated work. Note that the return statement (and all the code that potentially exists before it but within the function) is tabulated. This is Python's way of indicating which statements belong within a function's body and which ones do not.

Once you have defined a function, you can call it by writing its name, always followed by parentheses, and (optionally) passing inputs called *arguments* that are associated with the parameters. For our example above, that could look like:

```
greet('Cecil')
```

which would return "Hello, Cecil". Once you receive a returned value, it's your call what you want to do. For example, you could have stored it in a variable to use it down the line, e.g.:

```
greeting = greet('Cecil')
```

would result into the variable `greeting` storing the string `'Hello, Cecil'`.

You could also pass the returned value into another function. For example,

```
print(greet('Cecil'))
```

would have printed `'Hello, Cecil'` into the console.

Now that you have an understanding of functions, you can see that the `assignment3.py` is organized into eight functions. Each one applies a bitwise operation on the binary values of its input(s) and returns the result. Before we discuss what these bitwise operations are, we need to see what bitwise operators are supported as is in Python.

2.2 Bitwise operators in Python

Python supports six bitwise operators (`<<`, `>>`, `&`, `|`, $\sim$, and $\wedge$) that are applied on the individual bits of a number written in two's complement binary.

Here is a detailed explanation for each of the bitwise operations that Python supports but don't forget to read the Quick Tips at the bottom of the section, too.

- `x << y`: Left shift. Note that because Python doesn't have a fixed number of bits for integers, we don't discard the first y bits. Instead, we fill out `y` zeros on the right. This is the same as multiplying `x` by `2**y`. For example, if `x=47`, then `x << 2 == 47 * (2**2) == 188`. If we see it in binary, that would be shifting the binary number `0101111` by adding two zeros to the right, resulting in `010111100`. Similarly for negative numbers, if `x=-47`, then `x << 2 == (-47) * (2**2) == -188`. If we see it in binary (two's complement), that would be shifting the binary number `1010001` by adding two zeros to the right, resulting in `101000100`.
- `x >> y`: Arithmetic right shift. Returns `x` with the bits shifted to the right by `y` places. The last `y` bits are discarded and `y` bits matching the sign bit (1 for negative, 0 for positive) are added to the left. This is the same as the *floor division* of `x` by `2**y`. For example, for `x=47`, shifting `x >> 2 == 47 // (2**2) = 11`. Or seen in binary, this would be shifting the number `0101111` two bits to the right, discarding the last two digits (11) and adding two 0s (because it's a positive number) at the beginning resulting in `0001011` or `1011` for short. Similarly for negative numbers, if `x=-47`, then `x >> 2 == (-47) // (2**2) == -12`. If we see it in two's complement, that would be shifting the binary number `1010001` and discarding the last two bits (01) and adding two ones to the left, resulting in `1110100` or `10100` for short in two's complement.
- `x & y`: Bitwise and. Each bit of the output is 1 if the corresponding bit of `x` AND of `y` is 1, otherwise it's 0. For example, if `x=3` and `y=5`, then `x & y = 1` because `x` in binary is `0011` and `y` in binary is `0101`. Thus, applying the and operator on each bit, we get in binary 0001.

- `x | y`: Bitwise or. Each bit of the output is 0 if the corresponding bit of `x` AND of `y` is 0, otherwise it's 1. For example, if `x=3` and `y=5`, then `x | y = 7`. Remember that `x` in binary is `0011` and `y` in binary is `0101`. Thus, applying the or operator on each bit, we get in binary 0111.

- $\sim$`x` Returns the complement of `x`, that the number you get by switching each 1 for a 0 and each 0 for a 1. This is the same as `-x-1`. For example, if `x=47`, then $\sim$`x` results to -48.

- `x` $\wedge$ `y`: Bitwise xor. Each bit of the output is 1 if the corresponding bit at `x` and `y` are different and 0 if they are the same. For example, if `x=3` and `y=5`, then `x` $\wedge$ `y = 6`. Remember that `x` in binary is `0011` and `y` in binary is `0101`. Thus, applying the xor operator on each bit, we get in binary 0110.

Quick tips:

- Python does not have a fixed number of bits for integers. (It used to, and many languages, such as C++ and Java, do fix the number of bits for integers, typically to 32.)
- For the bitwise arithmetic to work, when `x` and `y` don't have the same number of bits, Python prepends the smaller one with 0s if it is a positive number, and with 1s if it is a negative number.
- If you have a positive number `x`, you can see its binary representation by calling `bin(x)`. For example, if `x=47`, then `bin(47)` would return `0b101111`. Ignore the 0b part which represents a positive binary number. What you got is that 47 in binary is 101111.
- Unfortunately, the trick to get the two's complement representation of a negative number is slightly more complicated. If you have a negative number `x`, you can see its binary representation in two's complement by calling `bin(x & 0b1111111111111111)`. For example, if `x=-47`, then `bin(x & 0b1111111111111111)` would return `0b1111111111010001`. Ignore the 0b. What you got is that -47 in binary two's complement is 1010001. Remember, sign extension for two's complement prepends the number with 1s
- If we have an integer `x`, we can extract its number of bits by calling `x.bit_length()`. The number you get back does not contain the sign.

2.3 Back to the Puzzles

We know what functions are, and we know which bitwise operators Python supports. We are now ready to discuss more about the eight puzzles you will work toward solving. The trick is that the puzzles will need to be completed using only straight-line code (no loops or conditionals, that is, if/else statements) and no function calls beyond calling `bit_length()`.

We have also imposed constraints on which bitwise operators Python supports, you can use to solve each puzzle. Within a function that corresponds to a puzzle, you are allowed to use local variables if your solution is complex and cannot be contained in a single line. Each function's *docstring* (documentation string), whose start and end are indicated with three straight single

quotes, explains what each function does, which operators you are allowed to use, and how many of them at most.

At the bottom of the file, you will see eight calls to functions, one for each puzzle. You should use these print statements to at least test whether your code matches the suggested output in the comments. But you should also make at least one more call for each function to test it more thoroughly.

ATTENTION! After you are satisfied with your testing, comment out all calls to functions, including both ours and yours. This will ensure that the autograder can run properly.

3 Feedback

Create a file named `feedback3.txt` that answers the questions:

- How long did you spend on this assignment?
- Any comments or feedback? What things did you find interesting, challenging, or boring?

4 When you're done

Submit your `assignment3.py` and `feedback3.txt` online using Gradescope. Be sure you're uploading them to the right assignment. Only one submission will be made for both partners. Note that you can resubmit the same assignment as many times as you would like up until the deadline.

5 Grading

	points
`bitwise_and`	1
`bitwise_nor`	1
`bitwise_xor`	1
`are_equal`	1
`negate`	1
`is_even`	1
`absolute`	2
`least_significant_bit`	1
At least eight additional tests in comments	1
Total	10